Adaptive Mixture-of-Experts (MoE) Expert Offloading

Review 1 — Complete Project Report

Scope: This Review 1 report stops at the completed cross-workload evaluation. EAM and MoE-Infinity-style work are deliberately excluded from the completed-work story and are listed only as future work.

Project in one sentence: We are investigating whether future OLMoE expert activations can be predicted early enough to prefetch those experts into limited GPU memory, reducing expert offloading cost while balancing prediction accuracy, coverage and cache overhead.

1. Executive Summary

Mixture-of-Experts (MoE) language models contain many expert networks but activate only a small subset for each token. This reduces computation but creates a memory-management problem when GPU memory is limited. Our project investigates predictive expert offloading: observing OLMoE routing, identifying temporal structure, predicting future experts, and evaluating whether those predictions can improve a GPU/CPU/Disk cache.

The Review 1 work establishes the experimental foundation: real OLMoE execution, routing instrumentation, trace collection and validation, locality analysis, history-based prediction, layer-aware transition prediction, selective prediction, multi-tier cache simulation, and cross-workload evaluation.

The key result is not a claim that one predictor is universally best. Instead, the experiments demonstrate a clear **precision–coverage–overhead trade-off**. This motivates the next phase: evaluate a published-method-inspired baseline, decide whether a learned predictor is justified, develop an adaptive policy, and eventually measure real end-to-end OLMoE inference.

2. Problem and Motivation

What is MoE? A dense Transformer applies the same feed-forward network to every token. An MoE model has many experts and a router selects only a subset for each token.

```
Token → Router → Top-8 experts → Expert computation → MoE output
```

Our OLMoE configuration contains 16 layers, 64 experts per layer, and top-8 routing. Therefore:

Quantity	Calculation	Result
Expert locations	16 × 64	1,024
Expert selections/token	16 × 8	128
Routing records for N tokens	N × 16	16N
Expert selections for N tokens	N × 16 × 8	128N

Why offloading? The complete expert set is much larger than the constrained GPU-memory budget used in our experiments. We therefore model a hierarchy:

```
GPU cache (fast) → CPU memory → Disk (slow)
```

Why prediction? LRU and similar caches react after a miss. A predictor attempts to move likely-needed experts before the request. Correct predictions may avoid expensive fetches; wrong predictions can waste bandwidth, consume cache space and cause evictions.

3. Experimental Setup

Item	Configuration
Model	allenai/OLMoE-1B-7B-0924
Layers	16
Experts/layer	64
Experts/token/layer	8
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GB
Workloads	WikiText-103, GSM8K, MBPP
Software	Windows 11, Python venv, PyTorch/CUDA

The native OLMoE router remains authoritative. Our prediction mechanisms are an optimization layer around the router; they do not replace actual routing decisions.

Why three workloads? WikiText represents general text, GSM8K mathematical reasoning, and MBPP programming/code. Using multiple workloads helps determine whether routing behavior is workload-dependent.

4. Routing Trace Collection

For every token and every layer, we record sample ID, token position, layer ID, selected experts and routing weights.

```
{"sample_id":1,"token_position":0,"layer_id":0,
"selected_experts":[57,45,42,30,17,22,51,27],
"routing_weights":[...]}
```

An **expert activation trace** is the sequence of these layer-specific routing decisions. We first used 10 samples per workload for rapid development and debugging. After the pipeline was stable, we expanded to 100 samples per workload.

5. Expanded 100-Sample Dataset

Workload	Samples	Tokens	Routing records	Expert selections
WikiText	100	9,180	146,880	1,175,040
GSM8K	100	5,546	88,736	709,888
MBPP	100	1,800	28,800	230,400
Combined	300	16,526	264,416	2,115,328

The counts are mathematically consistent. For WikiText, 9,180 × 16 = 146,880 routing records and 146,880 × 8 = 1,175,040 selections. The GSM8K and MBPP counts satisfy the same identities.

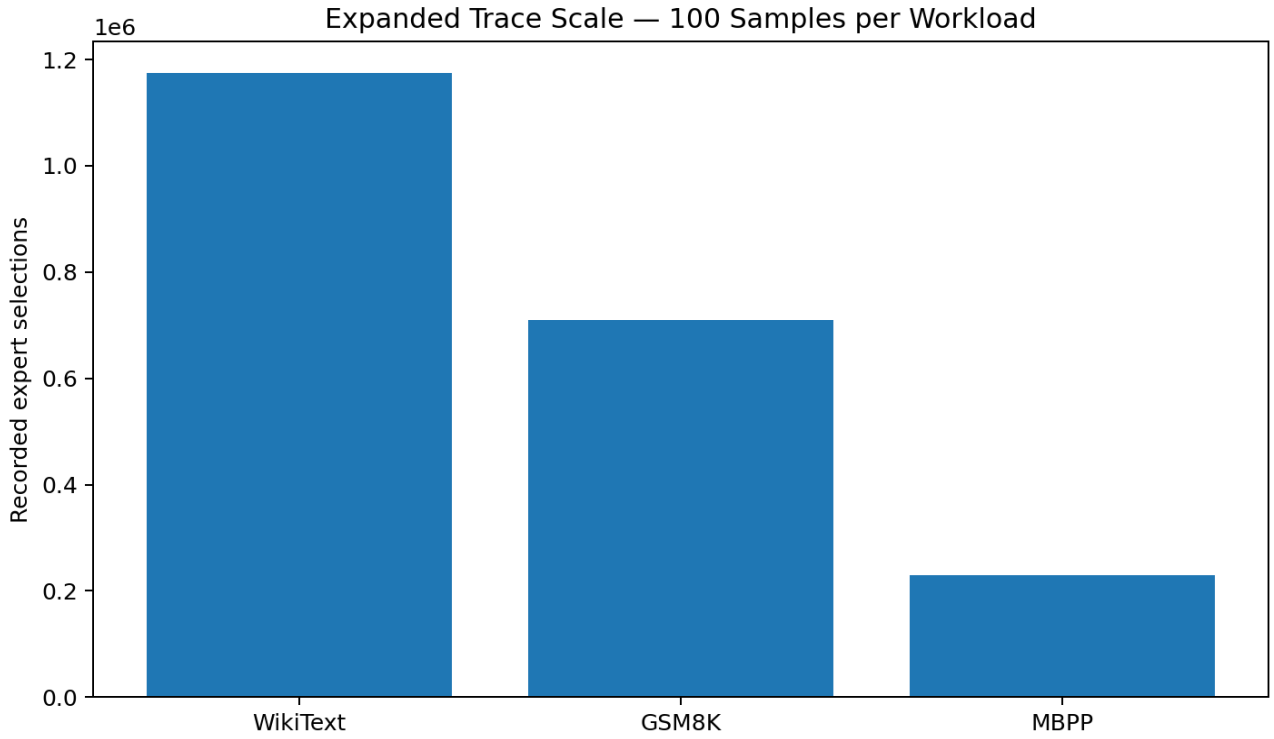


Figure 1. Expanded trace scale. Each workload contains 100 samples; the bars show the number of recorded expert selections. This is a data-scale figure, not a performance result.

Trace validation: all three expanded traces passed checks for 100 sample IDs, required fields, valid layers 0–15, valid experts 0–63, exactly 8 experts and 8 weights per event, no duplicate experts within an event, complete 16-layer coverage per token, and the routing-record formula.

6. Train/Test Preparation

The expanded traces were split at the sample level using an 80/20 split. This avoids putting routing records from the same input into both train and test sets.

Workload	Train	Test	Overlap
WikiText	80	20	0
GSM8K	80	20	0
MBPP	80	20	0

This split is ready for later learned-predictor evaluation. The strategy/cache performance numbers in this Review 1 report remain the earlier 10-sample preliminary results.

7. Locality Analysis — Why Prediction Is Plausible

Before building predictors, we asked whether previous routing decisions contain information about future routing. If expert activation were completely random, recent history would have little predictive value.

Lookback window	Hit rate
1	0.27%
2	0.88%
4	3.39%
8	20.31%

Lookback window	Hit rate
16	41.17%
32	57.58%
64	74.77%

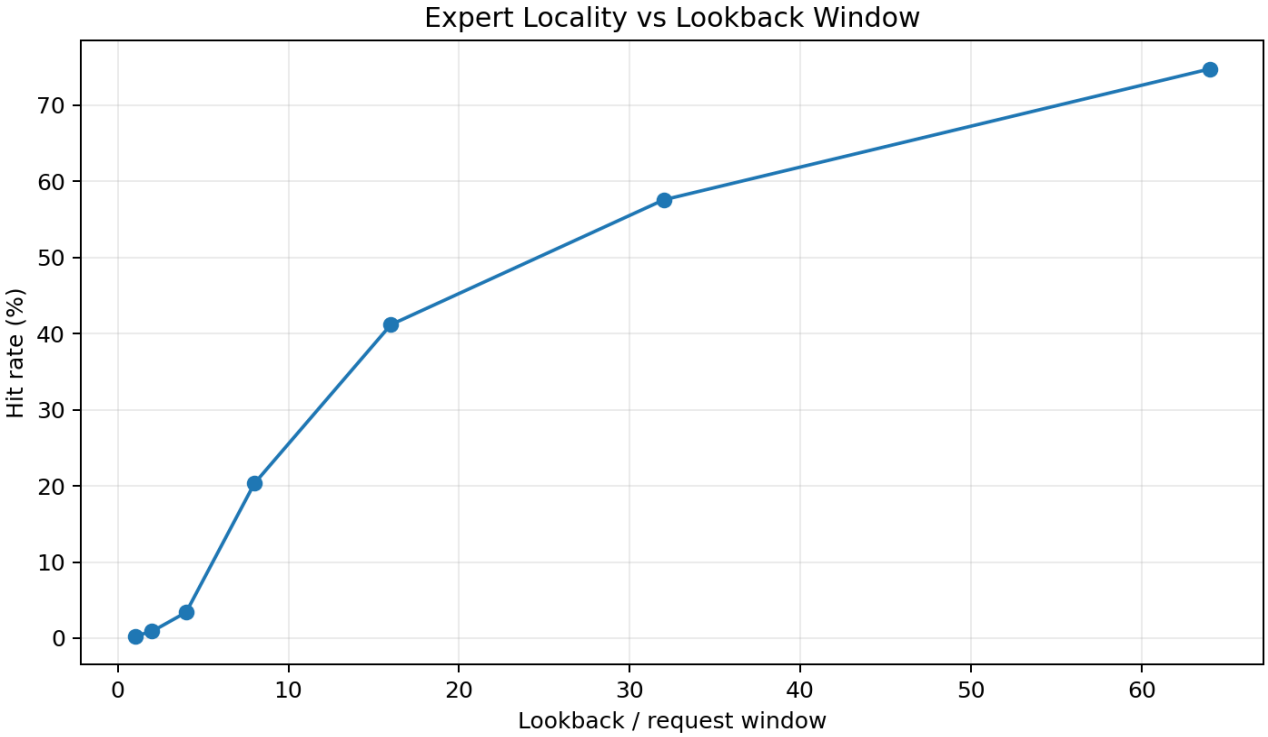


Figure 2. Expert locality versus lookback window. Preliminary WikiText result. The curve rises strongly as more history is considered.

Interpretation: immediate reuse is low, but longer history contains substantial information. At a 64-request lookback the hit rate reaches 74.77%.

Mathematical consistency: a larger lookback contains the smaller window plus additional history, so the candidate set cannot shrink. Under the same hit definition, a non-decreasing hit-rate curve is expected. Our measured sequence is monotonic: 0.27 < 0.88 < 3.39 < 20.31 < 41.17 < 57.58 < 74.77.

Neighboring tokens also shared an average of 2.87 of 8 selected experts, or approximately 35.83% overlap. An 8-token lookback covered about 75.46% of expert selections in the preliminary locality analysis.

8. Predictor 1 — History-Based Prediction

The History predictor stores recent token-level expert sets and ranks experts by recent frequency.

```
Recent expert sets → frequency counts → ranked candidates → top-N prediction
```

This is a deliberately simple baseline. It answers how much predictive value can be obtained from recent temporal reuse alone.

9. Predictor 2 — Layer-Aware Transition Prediction

The Transition predictor models which experts tend to occur after currently active experts. For each layer it maintains transition counts C_l(i,j).

```
Current experts → layer-specific transitions → candidate scores → ranked future experts
```

Layer awareness is essential because (layer 2, expert 5) and (layer 10, expert 5) are different parameter locations. The cache uses the same (layer_id, expert_id) identity.

10. Predictor 3 — Selective Transition Prediction

Predicting more experts increases the chance of covering future activity, but also increases wrong predictions and cache overhead. Selective prediction filters transition candidates using a relative transition-confidence threshold.

Important terminology: relative transition confidence is a normalized transition score, not a calibrated probability.

11. Evaluation Metrics and Measurements

Metric	Formula / definition	What it answers
Precision	Correct / predicted × 100	How accurate are the predictions we made?
Coverage	Correct / actual future selections × 100	How much future activity did we capture?
Hit	Requested expert already in the relevant cache tier	Was the expert already available?
GPU hit	Expert already on GPU	Fastest access path
CPU hit	Expert in CPU tier	Requires CPU→GPU transfer
Disk fetch	Expert absent from GPU and CPU	Most expensive modeled access
Prefetch	Expert moved before request	Was prediction used proactively?
Eviction	Cached expert removed	How much cache pressure occurred?
Utilization	Useful prefetches / issued prefetches × 100	How much prefetch work was useful?
Wasted prefetch	Issued prefetch not used as intended	How much prediction work was wasted?
Average modeled latency	Total modeled cost / expert requests	What is the modeled system cost?

Precision versus coverage: high precision can be obtained by making very few predictions. High coverage generally requires predicting more candidates. Therefore neither metric alone is sufficient.

12. Selective Prediction Results

Threshold	Precision	Coverage
0.000	41.17%	41.10%
0.025	41.26%	41.05%
0.050	56.07%	20.85%
0.075	71.37%	6.67%
0.100	83.68%	2.11%
0.105	85.07%	1.41%

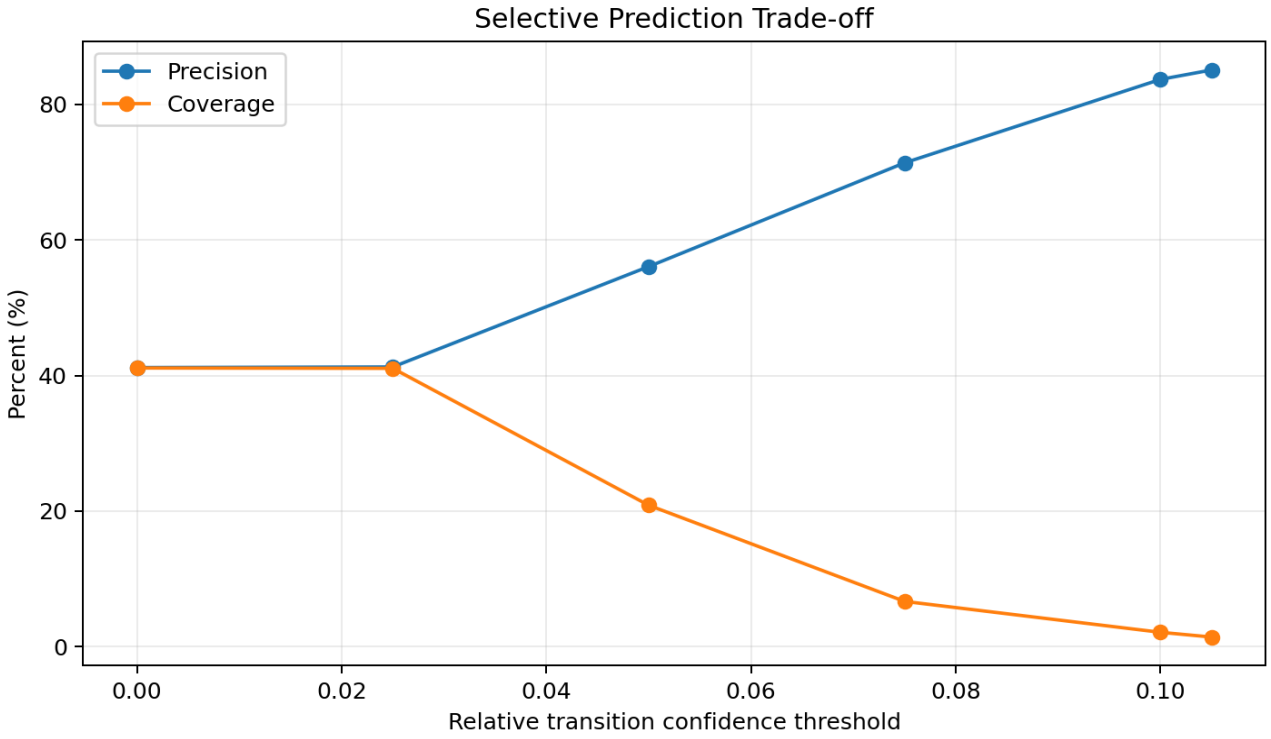


Figure 3. Selective prediction trade-off. Increasing the relative transition-confidence threshold raises precision while reducing coverage.

The relationship is mathematically consistent: raising the threshold removes lower-confidence candidates, so the prediction set shrinks. If the removed candidates are less accurate, precision rises. Because fewer candidates remain, coverage falls.

13. Multi-Tier Cache and Cost Model

The predictors were connected to a layer-aware GPU/CPU/Disk cache. The constrained configuration used 256 GPU expert slots and 170 CPU expert slots, while the system contains 1,024 layer-specific expert locations.

```
GPU cache → CPU memory → Disk
```

Operation	Modeled cost
GPU hit	0.05 ms
CPU → GPU	1.0 ms
Disk → GPU	10.0 ms
Prefetch	1.0 ms

The modeled total cost is conceptually:

```
T_total = N_GPU·C_GPU + N_CPU·C_CPU + N_Disk·C_Disk + N_Prefetch·C_Prefetch
```

Average modeled latency = total modeled cost / number of expert requests. These values are controlled simulation costs, **not real end-to-end OLMoE inference measurements**.

14. Unified Preliminary WikiText Comparison

Strategy	Precision	Coverage	Disk fetches	Avg modeled latency
LRU	—	—	6,675	0.4699 ms
History	31.83%	31.80%	5,948	0.4870 ms
Transition	41.17%	41.10%	5,664	0.5063 ms

Strategy	Precision	Coverage	Disk fetches	Avg modeled latency
Selective	85.07%	1.41%	6,648	0.4701 ms

These are **preliminary 10-sample WikiText results**. LRU is the no-prediction baseline.

Transition reduces disk fetches from 6,675 to 5,664, a reduction of approximately: (6,675 – 5,664) / 6,675 × 100 ≈ **15.15%**. However, modeled latency is not lower because prediction and cache effects also contribute to total cost.

15. Main System-Level Graph — Disk Fetches

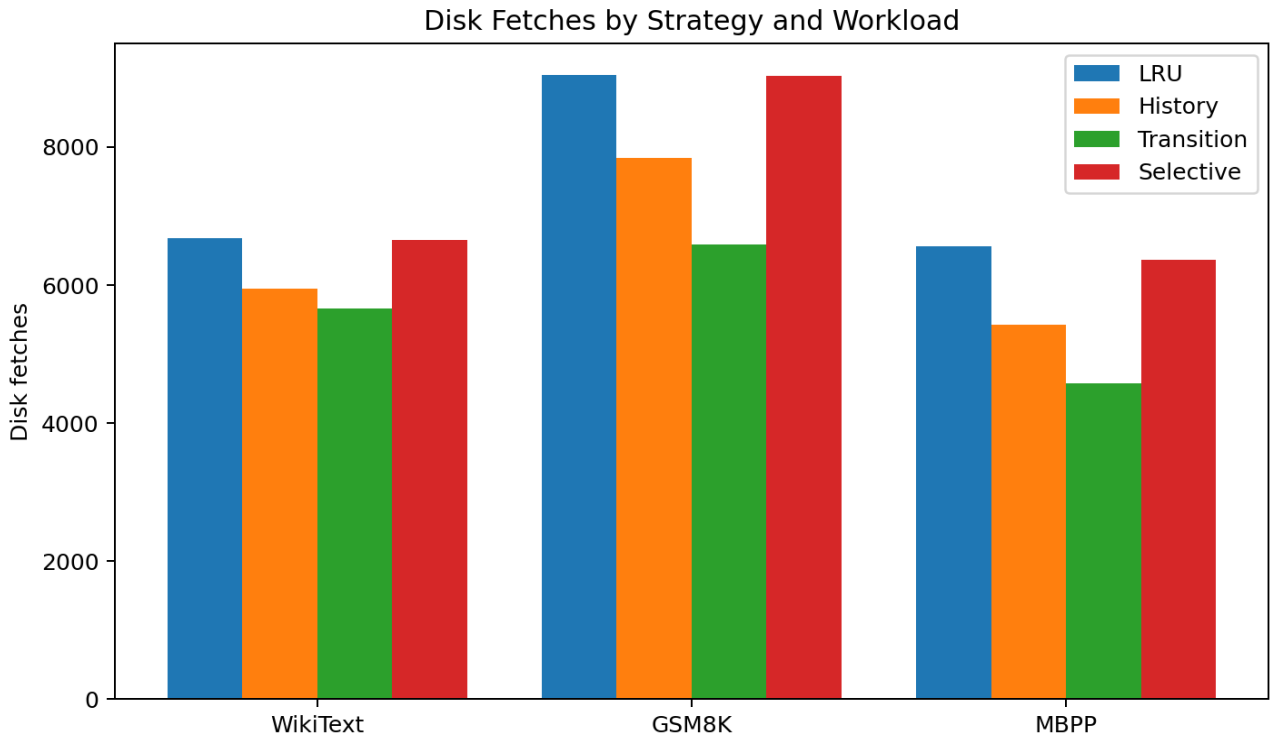


Figure 4. Disk fetches by strategy and workload. Lower disk fetch count means fewer modeled disk-level retrievals. The graph uses preliminary 10-sample results.

The graph shows that History and Transition reduce disk fetches relative to LRU in all three preliminary workloads. Transition produces the largest reduction among the prediction strategies in these experiments. Selective remains close to LRU because its high threshold produces very few predictions.

Why this matters: disk fetches are modeled as the most expensive access tier. Reducing them is therefore a useful intermediate system metric, although it does not by itself prove lower end-to-end latency.

16. Main System-Level Graph — Modeled Latency

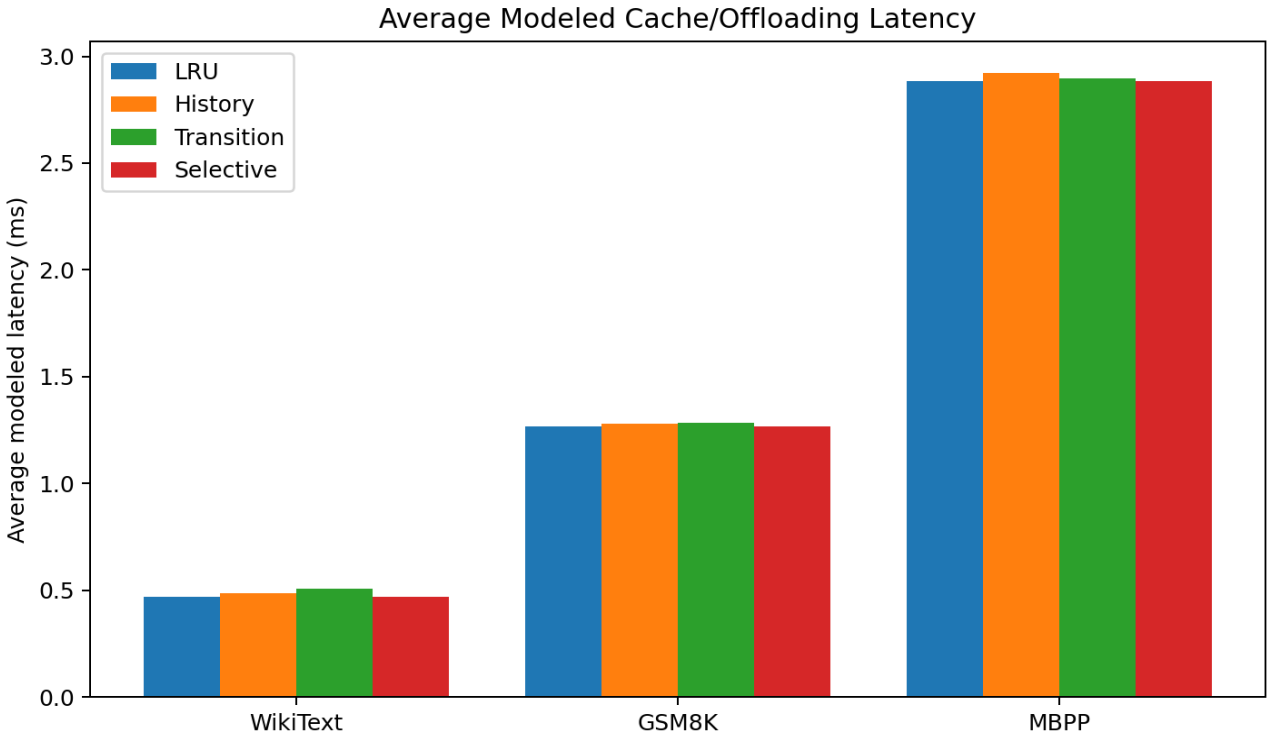


Figure 5. Average modeled cache/offloading latency. These are simulation values, not real end-to-end OLMoE inference measurements.

The bars are relatively close. This is important rather than a problem to hide: prediction reduces some expensive disk accesses but also introduces prefetch work and may change cache/eviction behavior. Therefore total modeled latency depends on the entire cost equation, not disk fetches alone.

17. Cross-Workload Evaluation

Workload	History P/C	Transition P/C	Selective P/C
WikiText	31.83 / 31.80%	41.17 / 41.10%	85.07 / 1.41%
GSM8K	31.87 / 31.82%	39.42 / 39.26%	76.84 / 1.30%
MBPP	43.11 / 42.87%	55.58 / 54.94%	90.97 / 19.51%

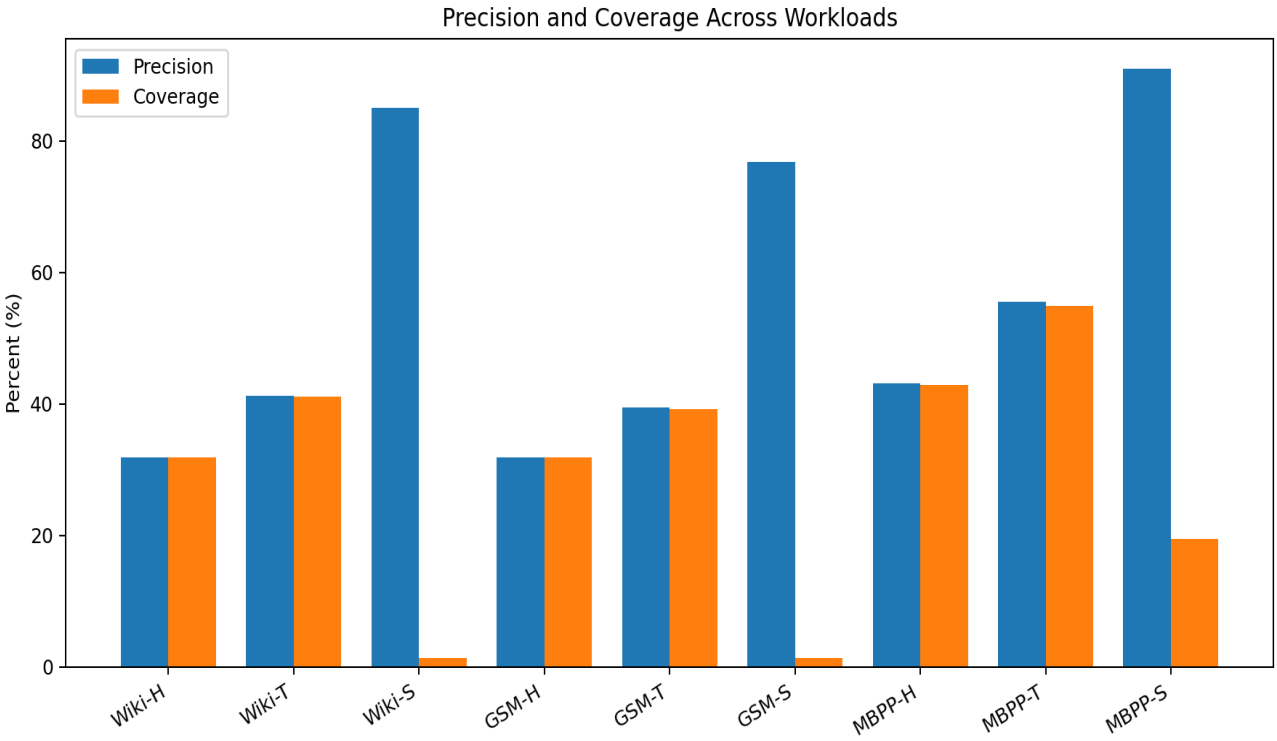


Figure 6. Precision and coverage across workloads and strategies. Abbreviations: H = History, T = Transition, S = Selective. Preliminary 10-sample results.

The broad trade-off is visible across all three workloads. Transition generally has higher precision and coverage than History in these preliminary experiments. Selective has substantially higher precision but much lower coverage.

MBPP has higher observed Transition precision/coverage than WikiText and GSM8K in this particular preliminary sample. We should not generalize this to all code workloads without the larger evaluation.

18. Why the Results Are Consistent

• **Routing counts:** 16 layers × 8 selected experts = 128 expert selections per token; all trace totals satisfy this identity.

• **Locality:** larger lookback windows contain the smaller windows plus additional history, so the measured hit rate is expected to be non-decreasing.

• **Selective prediction:** higher thresholds produce smaller prediction sets. This can raise precision while reducing coverage.

• **Cache cost:** disk access is modeled as much more expensive than GPU access, but prefetch itself has a cost. Therefore fewer disk fetches do not guarantee lower total modeled latency.

• **Cross-workload differences:** different inputs produce different token sequences and hidden states, which can lead to different router behavior. The preliminary results therefore need workload-aware interpretation.

19. What Has Been Completed

• Real OLMoE model execution and router instrumentation.

• WikiText-103, GSM8K and MBPP routing traces.

• Initial 10-sample traces for development and debugging.

• Expanded 100-sample traces: 300 samples and 2,115,328 expert selections combined.

• Trace validation and mathematical consistency checks.

• 80/20 sample-level train/test split with zero sample overlap.

• Expert locality and token-level overlap analysis.

• History-based predictor.

• Layer-aware transition predictor.

• Selective transition predictor.

• Layer-aware GPU/CPU/Disk cache.

• Cost model and prefetch simulation.

• Precision, coverage, disk-fetch and modeled-latency evaluation.

• Unified comparison and cross-workload preliminary evaluation.

20. What Is Still Left

```
REVIEW 1 BOUNDARY
Real OLMoE → Traces → Locality → History → Transition → Selective
→ Multi-tier cache → Unified comparison → Cross-workload evaluation
→ REVIEW 1

NEXT PHASE
→ MoE-Infinity-style / published-method-inspired baseline
→ Fair comparison with current predictors
→ Decide whether learned predictor is justified
→ Learned predictor if justified
→ Adaptive prefetch/offloading policy
→ Live OLMoE integration
→ Real end-to-end evaluation
→ Final ablations, graphs and report
```

A neural predictor is therefore not assumed to be necessary. Its inclusion should be justified by an unresolved precision/coverage/overhead limitation after the next baseline comparison.

21. Important Code Files to Show

Code	Purpose in the review
OLMoE trace collector	Shows actual router instrumentation.
scripts/check_gpu.py	GPU/CUDA verification.
scripts/validate_large_traces.py	Trace correctness and count checks.
scripts/create_train_test_split.py	Leakage-safe sample split.
src/prefetch/history_prefetcher.py	History baseline.
src/prefetch/transition_prefetcher.py	Layer-aware transition model.
src/prefetch/selective_prefetcher.py	Confidence filtering.
src/cache/expert_cache.py	Layer-aware LRU cache.
src/cache/cost_model.py	Modeled costs.
src/cache/multi_tier_cache.py	GPU/CPU/Disk hierarchy.
scripts/run_strategy_comparison.py	Unified evaluation.
scripts/generate_review1_graphs.py	Graph generation for review.

If the panel asks for only five files: show the routing collector, transition_prefetcher.py, selective_prefetcher.py, multi_tier_cache.py and run_strategy_comparison.py.

22. Likely Reviewer Questions

Why not keep all experts on GPU? The experiment assumes a constrained 4 GB GPU, while there are 1,024 layer-specific expert locations.

Why LRU? It is a standard reactive baseline without prediction.

Why History? It measures the value of simple temporal reuse.

Why Transition? It models relationships between current and subsequent expert activations.

Why Selective? It controls prediction volume and exposes the precision–coverage trade-off.

Why is Selective's precision high? A high threshold removes lower-confidence candidates.

Why isn't high precision automatically enough? At threshold 0.105, precision is 85.07% but coverage is only 1.41%. It predicts very little future activity.

Are the latency numbers real? No. They are modeled cache/offloading values. Real end-to-end OLMoE inference is future work.

Why three workloads? To examine workload dependence.

Why no neural network yet? We first establish simpler baselines and a published-method-inspired comparison; a learned predictor should be justified by the remaining gap.

Does the predictor replace the OLMoE router? No. The native router remains authoritative.

23. Important Limitations — Be Explicit in Review

• The strategy/cache performance graphs are preliminary **10-sample** experiments.

• The 100-sample traces have been collected and split, but the full strategy comparison has not yet been rerun on those expanded traces.

• Current latency is **modeled cache/offloading latency**, not real end-to-end inference latency.

• 100 samples per workload do not justify claims about the entire source datasets.

• The selective confidence score is not a calibrated probability.

• The adaptive policy and live OLMoE integration are not complete.

24. What Not to Claim

• Do not claim a final neural predictor is complete.

• Do not claim real end-to-end latency reduction.

• Do not claim exact reproduction of MoE-Infinity.

• Do not call one strategy universally best.

• Do not equate prediction precision with inference-performance improvement.

• Do not claim full-dataset generalization.

25. Recommended Review 1 Conclusion

“In the first phase of the project, we established a complete experimental pipeline for adaptive expert offloading using a real OLMoE model. We instrumented the native router and collected expert activation traces from WikiText-103, GSM8K and MBPP. We analyzed expert locality and found measurable temporal structure, then implemented history-based, layer-aware transition and selective predictors. These predictors were evaluated with a GPU/CPU/Disk cache model using precision, coverage, disk fetches and modeled latency. The preliminary cross-workload results demonstrate a clear precision–coverage–overhead trade-off: predicting more candidates increases coverage but can reduce precision and add cache/prefetch cost, while highly selective prediction improves precision at the cost of coverage. We also expanded the

data to 100 samples per workload and created a leakage-safe 80/20 split. The next phase will evaluate a published-method-inspired baseline, determine whether a learned predictor is justified, develop an adaptive offloading policy, and finally integrate the approach with live OLMoE inference for real end-to-end evaluation.”

> **Easy project summary:** observe routing → find structure → predict experts → prefetch experts → measure whether prediction actually helps.

Appendix A — Formula Sheet

```
Expert locations = 16 × 64 = 1,024
Selections/token = 16 × 8 = 128
Routing records = tokens × 16
Expert selections = tokens × 16 × 8
Precision = Correct / Predicted × 100
Coverage = Correct / Actual future selections × 100
Average modeled latency = Total modeled cost / Expert requests
Disk-fetch reduction = (Baseline − Strategy) / Baseline × 100
```

Example: Transition WikiText disk-fetch reduction = (6,675 – 5,664) / 6,675 × 100 ≈ 15.15%.

Appendix B — Recommended Presentation Order

```
1. Problem and motivation
2. MoE / OLMoE architecture
3. Why offloading is needed
4. Hardware and experimental setup
5. Routing instrumentation
6. Workloads
7. Trace validation and 100-sample scale
8. Locality result + graph
9. History predictor
10. Transition predictor
11. Selective predictor + trade-off graph
12. Metrics
13. Cache architecture
14. Disk-fetch graph
15. Modeled-latency graph
16. Cross-workload graph
17. Completed work
18. Limitations
19. Future work
20. Conclusion
```